

Compte rendu de la 1^{ère} séance de « TP »

de Programmation Orientée Objet sur Link

Dans ce TP, l'objectif était de commencer à construire les bases d'un petit moteur graphique 2D en Java. L'idée n'était pas encore de développer un jeu complet, mais plutôt de comprendre comment organiser un projet de manière propre en programmation orientée objet. Le but principal était donc d'ouvrir une fenêtre graphique, d'afficher un sprite et de mettre en place une architecture qui pourra être étendue plus tard avec un moteur physique, une carte de jeu ou encore la gestion des collisions. Au cours du TP, j'ai progressivement construit plusieurs classes qui ont chacune un rôle précis dans l'architecture du programme. Cette séparation des responsabilités permet d'éviter d'avoir tout le code dans une seule classe et rend le projet beaucoup plus évolutif.

Le programme repose sur une idée assez simple : une fenêtre graphique contient un moteur de rendu. Ce moteur de rendu est chargé d'afficher tous les objets présents dans la scène. Ces objets sont représentés par des sprites. Certains sprites sont simples, d'autres peuvent être animés. L'ensemble du système repose donc sur une séparation entre la logique du moteur et les objets affichés. C'est pour cette raison que j'ai utilisé des interfaces comme Displayable afin que le moteur de rendu puisse manipuler différents types d'objets sans avoir besoin de connaître leur implémentation exacte.

Ce compte-rendu ne sera pas très technique mais va résumer dans les grandes lignes ce que j'ai fait pour chaque classe et à quoi elles contribuent exactement.

La première structure importante que j'ai introduite est l'interface Engine. L'idée derrière cette interface est d'imposer une méthode « update() » commune à tous les moteurs du programme. Cette méthode sera appelée régulièrement afin de mettre à jour l'état du moteur. Dans le cas du moteur de rendu, cette mise à jour déclenche simplement un nouvel affichage. J'ai choisi de créer cette interface dès le début car cela permet d'imaginer facilement l'ajout d'autres moteurs dans le futur, par exemple un moteur de physique ou un moteur de jeu. En utilisant une interface, on garantit que tous les moteurs auront une structure similaire.

L'interface Displayable joue un rôle central dans l'architecture. Elle définit simplement une méthode « draw » qui reçoit un objet Graphics. Tous les objets qui peuvent être affichés à l'écran doivent implémenter cette interface. Le moteur de rendu peut donc parcourir une liste d'objets Displayable et appeler la méthode « draw » sur chacun d'eux. Ce mécanisme repose sur le polymorphisme : le moteur ne sait pas exactement quel type d'objet il dessine, mais il sait qu'il possède une méthode « draw ». Cette approche permet d'ajouter facilement de nouveaux types d'objets graphiques sans modifier le moteur de rendu.

La classe `RenderEngine` est responsable de l'affichage des objets à l'écran. Elle hérite de `JPanel` afin d'être intégrée dans le système graphique de Swing. J'y ai défini une liste d'objets `Displayable` appelée `renderList`. Chaque objet présent dans cette liste sera affiché lors du rendu. La méthode principale de cette classe consiste à redéfinir la méthode d'affichage afin de parcourir la liste et d'appeler `draw` sur chaque élément. Un point important que j'ai compris est l'utilisation de la méthode « `repaint()` ». Plutôt que de dessiner directement dans une boucle infinie, Swing recommande d'utiliser `repaint` qui demande au système graphique de redessiner le composant. Cette méthode est appelée régulièrement grâce à un `Timer`.

La classe `Sprite` représente un objet graphique de base. Elle contient une image ainsi que des informations de position et de taille. Le rôle principal de cette classe est de fournir une implémentation simple de la méthode « `draw` » qui affiche l'image à l'écran. Cette classe sert de base pour toutes les autres classes de sprites. En centralisant ces informations dans une classe parent, on évite de répéter le même code dans plusieurs classes.

La classe `SolidSprite` hérite directement de `Sprite`. Elle représente un sprite qui pourra être considéré comme un objet solide dans le jeu. Pour l'instant, elle ne possède pas encore de comportement spécifique, mais elle permettra plus tard d'identifier les objets qui doivent participer aux collisions. L'idée était donc d'anticiper les besoins futurs du projet en créant une hiérarchie de classes adaptée.

J'ai également utilisé « `Direction` » afin de représenter les différentes directions d'animation du personnage. Chaque direction correspond à une ligne spécifique dans la sprite sheet. En stockant directement cette information dans l'énumération, j'évite d'avoir des conditions complexes dans le code du sprite dynamique.

La classe `DynamicSprite` est une extension de `SolidSprite` qui permet d'afficher un sprite animé. Contrairement à `Sprite` qui affiche une image complète, cette classe découpe une sprite sheet contenant plusieurs images du personnage. Le principe consiste à sélectionner une portion précise de l'image source afin d'afficher une seule frame à la fois. Pour cela, j'utilise les coordonnées source dans la méthode « `drawImage` ». Un calcul basé sur le temps permet également de changer automatiquement la frame affichée afin de créer une animation fluide.

On pourrait croire que tout s'est passé comme prévu pendant la séance, mais en fait ce n'est pas le cas, j'ai eu un problème durant mon travail, cela concernait l'affichage de plusieurs sprites simultanément alors que je souhaitais n'en afficher qu'un seul. Après analyse, je me suis rendu compte que le problème venait du fait que toute la sprite sheet était affichée au lieu d'une seule frame. Pour corriger ce problème, j'ai modifié l'implémentation afin d'utiliser la classe `DynamicSprite`, qui est conçue pour gérer les animations. Cette classe ne se contente pas d'afficher l'image complète : elle sélectionne une portion précise de l'image source correspondant à une seule frame de l'animation. La correction principale a donc consisté à utiliser la version de `drawImage` qui permet de spécifier une zone source dans l'image. Cette zone est définie par les coordonnées `sx1`, `sy1`, `sx2` et `sy2`. Ces coordonnées correspondent à la position de la frame dans la sprite sheet. Une autre modification importante a été de définir correctement les dimensions d'une frame. Au départ, j'utilisais des valeurs de largeur et de hauteur trop grandes, ce qui provoquait un découpage incorrect de l'image. En utilisant les dimensions réelles d'une frame de la sprite sheet, le problème d'affichage a été corrigé. Après ces modifications, le moteur de rendu n'affiche plus l'ensemble de la sprite sheet mais uniquement la portion correspondant à la frame sélectionnée, ce qui permet d'obtenir l'affichage correct d'un seul sprite animé, ce qui était l'objectif que je m'étais fixé.

En conclusion, cette séance m'aura permis de mieux comprendre comment organiser un moteur graphique simple en Java en séparant clairement les différentes responsabilités du programme. La mise en place des classes de sprites m'a notamment permis de comprendre concrètement comment fonctionne l'affichage d'un sprite et l'utilisation d'une sprite sheet pour l'animation. Les difficultés rencontrées m'ont permis de progresser dans la maîtrise du langage, mais aussi de certaines classes qui me paraissaient obscures, que je « croyais » avoir comprises alors qu'au final cela se soldait par des erreurs d'affichages. La prochaine séance consistera à le faire se déplacer sur une map spécifique.